Nexus HMS User Management and Security Guide

Nexus HMS Enterprise Edition

Document control

- Product: Nexus HMS Enterprise Hotel Management System
- Release baseline: Phase 7B Enterprise Platform
- Document type: Administrator and security operations guide
- Date: 27 August 2026

1. Administrative model

Nexus HMS separates platform scope from tenant scope. A platform administrator has a null tenant context and may manage platform-wide configuration. A tenant administrator operates inside one tenant and may manage users, records, and security rules allowed by that tenant. Hotel roles should be assigned according to least privilege.

The client interface may hide modules based on role, but the server guards are the final authorization boundary. Never rely on a hidden navigation item to protect data.

2. User lifecycle

Provision

1. Confirm the user’s employment, department, tenant, and required access.
2. Create the account with the minimum role needed.
3. Confirm the email and contact information through the approved onboarding process.
4. Give the user a first-login instruction through a secure channel.
5. Require MFA for administrators and any role that handles sensitive guest, financial, or security information.

Change

Review role, tenant, department, and active status when the user changes duties. Remove access that is no longer needed before granting new access.

Suspend and remove

Deactivate users promptly when employment ends, access is compromised, or a device is lost. Revoke active mobile devices. Preserve audit records according to hotel retention policy rather than deleting evidence.

3. Roles and least privilege

Use role names that map to real responsibilities:

- Platform admin: tenants, subscriptions, modules, plugins, global password policy, and global security.
- Tenant admin: tenant users, local IP rules, hotel configuration, and permitted workflows.
- Manager: approvals, dashboards, reports, and operational oversight.
- Front desk: guests, reservations, check-in, stay, and checkout.
- Department operator: only the operational module needed by the department.
- Finance or HR specialist: sensitive finance or personnel operations.
- Read-only/reporting: view and export only what policy permits.

Review role assignments at least quarterly and after every organizational change. Avoid giving platform scope to a user who only needs one property's operational access.

4. Authentication controls

Access and refresh tokens

Access tokens authenticate normal API requests. Refresh tokens are separately signed and should be stored only in an appropriately protected client store. On logout, the client must remove stored tokens. If a refresh token is suspected to be compromised, follow the session invalidation and secret rotation procedure.

Production operations must provide matching ‘REDIS_PASSWORD‘ and credential-bearing ‘REDIS_URL‘ secrets for the cross-replica Socket.IO adapter. The Kubernetes API reads ‘REDIS_URL‘ from a Secret, while Redis reads ‘REDIS_PASSWORD‘ from the same Secret. NetworkPolicies permit Redis ingress only from API pods and deny Redis egress. Redis and web pods run explicitly as non-root without privilege escalation.

For real production, prefer managed Redis on a private network with TLS and use ‘rediss://‘. The Redis service included with Compose and Kubernetes requires authentication but does not enable TLS; do not treat its internal ‘redis://‘ connection as encrypted. Redis supports realtime packet delivery only; authorization remains enforced by the API and Redis is not a shared authorization cache or durable audit/event store.

MFA enrollment

1. The user opens Security and chooses MFA setup.
2. The server generates a TOTP secret and eight backup codes.
3. The QR data and codes are returned for one-time enrollment display.
4. The database stores encrypted values, not plaintext secrets.

5. The user enters a six-digit code.

6. A successful verification changes the MFA configuration to enabled.

Administrators should confirm that backup codes were stored safely without requesting that a user send them to support.

MFA recovery

An unused backup code can complete the MFA login step once. A consumed code cannot be reused. If all codes are lost, follow the organization’s identity-verification and administrator-assisted recovery process. Do not manually edit encrypted MFA database values.

Disablement

Disabling MFA requires the current TOTP code. This protects against a session being used to silently remove a second factor. If a user cannot produce the code, use the approved recovery process rather than bypassing the endpoint.

5. Password policy

Password policy is platform-wide and can be modified only by a platform administrator. Supported controls include:

- Minimum length, between 6 and 128 characters.

- Uppercase requirement.

- Number requirement.

- Symbol requirement.

- Expiry days, from no expiry through the configured maximum.

- Number of previous passwords to prevent reusing.

Choose policy values that match the organization’s risk assessment. Avoid policy changes during a login incident unless the change is part of the incident plan. A policy update should be recorded and communicated to administrators.

6. IP restrictions

IP rules support:

- Allowlist entries for an IPv4 address or IPv4 CIDR, such as ‘10.0.0.0/8‘.

- Blocklist entries for an IPv4 address or IPv4 CIDR.

- A label to explain the business purpose.

- Global rules managed at platform scope.

- Tenant rules managed within tenant scope by an administrator.

Before adding an allowlist rule, confirm the administrator’s current source IP and keep a tested recovery route. Invalid formats, IPv6 values, and malformed CIDR prefixes are rejected. When a rule changes, affected cached rules are invalidated.

Use network firewalls or private ingress controls as the primary perimeter. Application IP rules are a second layer and should not be the only protection for administrative access.

7. Device management

The Active Devices view lists active mobile sessions for the current user. Each device should have a recognizable name and platform. When a device is lost, replaced, or shared outside policy:

1. Revoke the device immediately.

2. Ask the user to change credentials if compromise is possible.

3. Check audit activity and recent login behavior.

4. Re-register only the verified replacement device.

8. Tenant and organization administration

Create a tenant for a customer boundary and organizations or branches according to the operating model. Validate tenant association before assigning users. Subscription and module configuration should be reviewed before exposing features to an end user.

Important current boundary: legacy core hotel tables do not yet include complete tenant_id row isolation. Tenant-aware BI and mobile services deliberately limit responses where a safe filter cannot be applied. Do not enable unrestricted cross-tenant operational access until the core-table migration is complete and verified.

9. Audit and incident response

Review audit logs for:

- Repeated failed logins or MFA failures.

- Role, tenant, or user status changes.

- MFA enrollment or disablement.

- IP rule mutations.

- Device registration and revocation.

- Password-policy changes.

- Cache flushes and unusual administrative actions.

For suspected compromise:

1. Deactivate the affected user or revoke the device.

2. Preserve audit and application logs.

3. Check active sessions, IP rules, and recent changes.

4. Rotate affected secrets using the environment secret process.

5. Determine whether MFA secrets, refresh tokens, Redis credentials, or database data were exposed.

6. Restore service only after verifying the new control state.

Do not include tokens, secrets, backup codes, or raw credentials in incident tickets.

10. Security administration checklist

- [ ] Platform and tenant administrators are documented.
- [ ] Production ‘SESSION_SECRET‘, ‘REDIS_PASSWORD‘, and the credential-bearing ‘REDIS_URL‘ are held in the environment secret store.
- [ ] Redis password and URL credentials match, and readiness/smoke checks authenticate.
- [ ] Redis is private, reachable by every API replica, restricted to API-only ingress with denied egress, and used only for the Socket.IO backplane.
- [ ] Real production uses managed/private TLS Redis with ‘rediss://‘, or an approved equivalent; the included internal Redis is not claimed to provide TLS.
- [ ] Administrator MFA is enabled and recovery ownership is known.
- [ ] Password policy has an approved owner and review date.
- [ ] Global and tenant IP rules have labels and recovery testing.
- [ ] Device revocation is part of offboarding.
- [ ] Metrics access is restricted at the network boundary.
- [ ] CORS origins are restricted in production.
- [ ] Backups are encrypted and restoration is tested.
- [ ] Audit logs are retained according to policy.
- [ ] Core-table tenant isolation limitations are acknowledged.